

Домашнее задание: Мониторинг

Курс: ClickHouse

Вариант выполнения: Вариант 1 - персонализированный мониторинг через встроенный dashboard ClickHouse.

Цель: настроить мониторинг и оптимизировать наблюдаемость ClickHouse с использованием пользовательских SQL-запросов.

1. Используемое окружение

Компонент	Значение
Платформа	Yandex Cloud Compute VM
Хост	compute-vm-2-4-30-ssd-1776951461260
Публичный IP	51.250.103.65
ClickHouse HTTP	8123
Выбранный вариант задания	Вариант 1: встроенный dashboard ClickHouse

2. Создание таблицы с пользовательскими запросами мониторинга

Для хранения пользовательских запросов была создана отдельная база monitoring и таблица monitoring.custom_dashboards. Таблица имеет структуру, совместимую с форматом встроенного dashboard ClickHouse: dashboard, title, query.

```
CREATE DATABASE IF NOT EXISTS monitoring;

DROP TABLE IF EXISTS monitoring.custom_dashboards;

CREATE TABLE monitoring.custom_dashboards
(
    dashboard String,
    title String,
    query String
)
ENGINE = MergeTree
ORDER BY (dashboard, title);
```

3. Набор запросов для персонализированного мониторинга

В dashboard были добавлены запросы для контроля текущей нагрузки, памяти, размеров таблиц, количества active parts, медленных запросов, ошибок и фоновых merge-операций.

N	Название	Назначение
1	Currently running queries	Количество выполняемых сейчас запросов из system.processes
2	Memory usage by running queries	Запросы с максимальным использованием памяти
3	Largest tables by size	Самые крупные таблицы по bytes_on_disk
4	Active parts by table	Количество активных parts по таблицам
5	Slow queries for last hour	Самые медленные запросы за последний час из system.query_log
6	Query errors for last hour	Ошибки запросов за последний час
7	Background merges	Текущие фоновые merge-операции

8	Current system metrics	Основные runtime-метрики из system.metrics
---	------------------------	--

Основной INSERT для наполнения таблицы:

```

INSERT INTO monitoring.custom_dashboards VALUES
('otus_custom_monitoring', '01. Currently running queries',
'SELECT now() AS time, count() AS running_queries FROM system.processes'),

('otus_custom_monitoring', '02. Memory usage by running queries',
'SELECT now() AS time, user, query_id, formatReadableSize(memory_usage) AS memory,
      elapsed, left(query, 120) AS query_text
FROM system.processes
ORDER BY memory_usage DESC
LIMIT 10'),

('otus_custom_monitoring', '03. Largest tables by size',
'SELECT database, table, formatReadableSize(sum(bytes_on_disk)) AS size_on_disk,
      sum(rows) AS rows, count() AS parts
FROM system.parts
WHERE active
GROUP BY database, table
ORDER BY sum(bytes_on_disk) DESC
LIMIT 10'),

('otus_custom_monitoring', '04. Active parts by table',
'SELECT database, table, count() AS active_parts, sum(rows) AS rows
FROM system.parts
WHERE active
GROUP BY database, table
ORDER BY active_parts DESC
LIMIT 10'),

('otus_custom_monitoring', '05. Slow queries for last hour',
'SELECT event_time, user, query_duration_ms, read_rows,
      formatReadableSize(read_bytes) AS read_bytes, left(query, 120) AS query_text
FROM system.query_log
WHERE event_time >= now() - INTERVAL 1 HOUR AND type = QueryFinish
ORDER BY query_duration_ms DESC
LIMIT 10'),

('otus_custom_monitoring', '06. Query errors for last hour',
'SELECT event_time, user, exception_code, exception, left(query, 120) AS query_text
FROM system.query_log
WHERE event_time >= now() - INTERVAL 1 HOUR AND type = ExceptionWhileProcessing
ORDER BY event_time DESC
LIMIT 10'),

('otus_custom_monitoring', '07. Background merges',
'SELECT now() AS time, database, table, elapsed, progress, num_parts,
      formatReadableSize(total_size_bytes_compressed) AS total_size
FROM system.merges
ORDER BY elapsed DESC
LIMIT 10'),

('otus_custom_monitoring', '08. Current system metrics',
'SELECT metric, value, description
FROM system.metrics
WHERE metric IN ('Query', 'Merge', 'MemoryTracking',
      'BackgroundPoolTask', 'ReadonlyReplica', 'DelayedInserts')
ORDER BY metric);

```

4. Проверка таблицы dashboard

После вставки запросов была выполнена проверка содержимого таблицы мониторинга.

```
SELECT
    dashboard,
    title
FROM monitoring.custom_dashboards
ORDER BY title;
```

Ожидаемый результат: таблица содержит набор записей с dashboard otus_custom_monitoring и названиями monitoring-запросов.

5. Настройка доступа к встроенному dashboard

Изначально ClickHouse HTTP-интерфейс слушал только localhost, поэтому внешний URL не открывался. Проверка порта показала прослушивание на 127.0.0.1:8123 и [::1]:8123.

```
sudo ss -lntp | grep 8123
```

```
# Было:
# LISTEN 0 4096 127.0.0.1:8123 0.0.0.0:* users:(("clickhouse-serv",...))
# LISTEN 0 4096 [::1]:8123 [::]:* users:(("clickhouse-serv",...))
```

Для доступа к dashboard с браузера был добавлен конфигурационный файл ClickHouse:

```
sudo tee /etc/clickhouse-server/config.d/listen.xml > /dev/null <<'EOF'
<clickhouse>
    <listen_host>0.0.0.0</listen_host>
</clickhouse>
EOF
```

```
sudo systemctl restart clickhouse-server
```

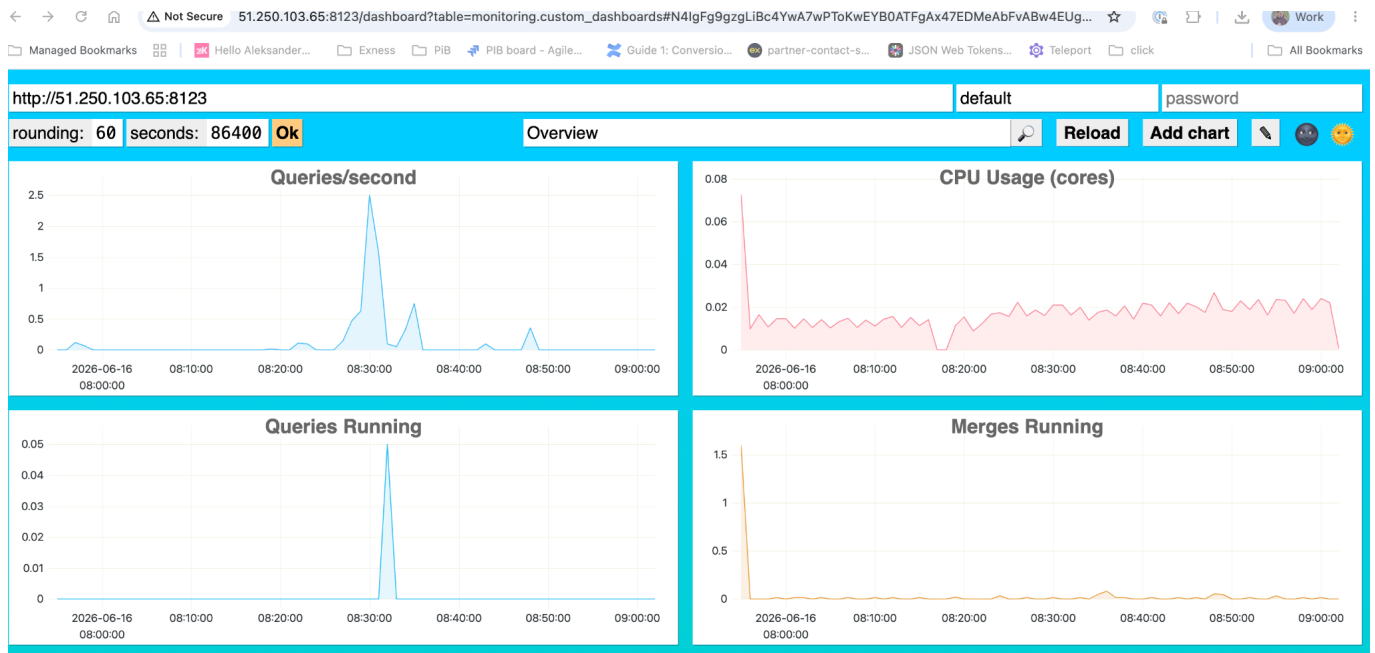
После этого был открыт порт 8123 в правилах сетевого доступа Yandex Cloud. Dashboard был открыт по адресу:

```
http://51.250.103.65:8123/dashboard?table=monitoring.custom_dashboards
```

Примечание по безопасности: после сдачи работы рекомендуется закрыть внешний доступ к 8123 или ограничить его своим IP-адресом.

6. Результат

На скриншоте ниже показан встроенный dashboard ClickHouse. Видны графики Queries/second, CPU Usage, Queries Running и Merges Running. Это подтверждает, что метрики ClickHouse собираются и отображаются во встроенном интерфейсе мониторинга.



7. Вывод

В ходе выполнения домашнего задания был выбран Вариант 1. Создана таблица `monitoring.custom_dashboards` с пользовательскими SQL-запросами для мониторинга ClickHouse. Настроен доступ к встроенному dashboard по HTTP-порту 8123. В результате dashboard успешно открылся и показал графики системных метрик ClickHouse. Цель задания достигнута.